IST602 — Web Technologies and Standards

June 2025 Examination — Model Answers

Course: IST602 – Web Technologies and Standards
Institution: University of Buea, Department of Computer Science
Exam date: 12th June 2025 (Semester 2024–2025)

Question 1 — URLs and HTTP (16 marks)

(a) Name the sections A–F of the URL (3 marks)

For: `http://www.mysite.com:80/path/to/mypage.html?product=camera#SomewhereInDoc`

Label	Component	Name
A	`http`	**Scheme** (or protocol) — how to access the resource
B	`www.mysite.com`	**Host** (authority / domain name)
C	`80`	**Port** — TCP port on the server (80 is default for HTTP)
D	`/path/to/mypage.html`	**Path** — location of the resource on the server
E	`?product=camera`	**Query string** — optional parameters for the server
F	`#SomewhereInDoc`	**Fragment** (fragment identifier) — location *within* the page (client-side only; not sent to server)

(b) HTTP protocol

(i) Request line and headers (4 marks)

Three parts of the HTTP request line (e.g. `GET /index.html HTTP/1.1`):

Part	Meaning
Method (`GET`)	What action the client wants (GET, POST, PUT, DELETE, etc.)
Request-URI (`/index.html`)	Target resource (path, and sometimes query)
HTTP version (`HTTP/1.1`)	Protocol version used for the message

Role of headers:

Header	Role
Host	Specifies the server hostname (required in HTTP/1.1 for virtual hosting)

Header	Role
User-Agent	Identifies the client (browser, bot, app)
Content-Type	MIME type of the message body (e.g. `application/json`)
Content-Length	Size of the body in bytes
Accept	Media types the client can handle in the response

(ii) How `Accept` is interpreted (2 marks)

Request 1: `Accept: text/html`
The client prefers **HTML** only. The server should ideally return `text/html` (or negotiate; if it cannot, it may return another type or an error depending on server behavior).

Request 2: `Accept: text/html, application/xhtml+xml, application/xml;q=0.9, */*;q=0.8`
The client sends a **priority list** with **quality values (q)**:

- Best: `text/html` and `application/xhtml+xml` (default q=1.0)
- Next: `application/xml` (q=0.9)
- Fallback: any type `*/*` (q=0.8)

The server chooses the **best matching** `Content-Type` the client will accept.

(iii) Delimiters in `GET /search?q=example&page=2` (1.5 marks)

Delimiter	Role
`?`	Starts the **query string** (separates path from parameters)
`=`	Separates a **parameter name** from its **value** (`q` = `example`)
`&`	Separates **multiple** query parameters (`q=example` and `page=2`)

(iv) Explain the POST request (2 marks)

```
POST /users HTTP/1.1
Host: example.com
Content-Type: application/json
Content-Length: 55

{ "name": "John Doe", "email": "john.doe@example.com" }
```

- **POST** to `/users` on `example.com` using **HTTP/1.1** — typically **create** a new user.
- **Content-Type: application/json** — body is JSON.
- **Content-Length: 55** — body size in bytes (header/body must match).
- Blank line separates headers from **body** (user data to be stored/processed).

(v) Three parts of the HTTP response status line (1.5 marks)

Example: `HTTP/1.1 201 Created`

Part	Meaning
HTTP version (`HTTP/1.1`)	Protocol version
Status code (`201`)	Numeric result (201 = created)
Reason phrase (`Created`)	Short human-readable description of the code

---

(vi) Explain the HTTP response (2 marks)

```
HTTP/1.1 201 Created
Content-Type: application/json

{ "message": "New user created", "user": { ... } }
```

- **201 Created** — request succeeded and a **new resource** was created.
- **Content-Type: application/json** — body is JSON.
- Body confirms success and returns the new **user** object (`id`, `firstName`, `lastName`, `email`).

Question 2 — HTML, CSS, JavaScript table (7 marks)

Task: Generate a table with columns **N**, **10×N**, **100×N** for N = 1…5; heading and body text **red**; body rows built with a `for` **loop**.

```
<!DOCTYPE html>
<html>
<head>
  <title>Table N, 10*N, 100*N</title>
  <style type="text/css">
    table { border: 1px solid black; border-collapse: collapse; }
    th, td { border: 1px solid black; padding: 4px; color: red; }
  </style>
  <script type="text/javascript">
    function buildTable() {
      var tbody = "";
      for (var i = 1; i <= 5; i++) {
        tbody += "<tr>";
        tbody += "<td>" + i + "</td>";
        tbody += "<td>" + (10 * i) + "</td>";
        tbody += "<td>" + (100 * i) + "</td>";
        tbody += "</tr>";
      }
      document.getElementById("tableBody").innerHTML = tbody;
```

```
    }
    window.onload = buildTable;
  </script>
</head>
<body>
  <table border="1">
    <thead>
      <tr>
        <th>N</th>
        <th>10*N</th>
        <th>100*N</th>
      </tr>
    </thead>
    <tbody id="tableBody"></tbody>
  </table>
</body>
</html>
```

Marking alignment:

Part	Requirement
(a)	Valid table structure; JavaScript generates 5 rows (1–5, 10–50, 100–500)
(b)	CSS sets `color: red` on `th` and `td`
(c)	`for` loop (`i` from 1 to 5) builds `<tr>` / `<td>` content

Question 3 — TCP/IP and TCP vs UDP (7 marks)

(a) Working of the TCP/IP protocol (5 marks)

TCP/IP is a **layered suite** of protocols used to send data across networks (especially the Internet). Data moves down the stack on the sender and up on the receiver.

Typical layers (conceptual):

1. **Application** — HTTP, DNS, SMTP, etc. (what the program needs).
2. **Transport** — **TCP** or **UDP** (end-to-end delivery between processes on hosts, using ports).
3. **Internet** — **IP** (routing packets between hosts using IP addresses).
4. **Network access / link** — Ethernet, Wi-Fi, etc. (frames on the local network).

How it works (summary):

- Application data is passed to **TCP** (or UDP).
- TCP splits data into **segments**, adds sequence numbers, ports, checksums; IP wraps segments in **packets** with source/destination IP addresses.
- Routers forward packets hop-by-hop using IP routing.
- At the destination, IP delivers to TCP; TCP **reorders**, checks integrity, and delivers a byte stream to the correct **port** and application.

- **DNS** resolves names to IPs; **HTTP** (etc.) runs on top once the connection exists.

Key idea: TCP/IP separates **"which application"** (port), **"which host"** (IP), and **"how to cross each network link"** (link layer).

(b) Contrast TCP and UDP (2 marks)

	TCP	**UDP**
Connection	Connection-oriented (handshake before data)	Connectionless (no setup)
Reliability	Reliable — retransmits lost data, ordered delivery	Unreliable — no guarantee of delivery or order
Overhead	Higher (headers, acknowledgments, flow control)	Lower, faster for small/simple messages
Use cases	Web (HTTP), email, file transfer — when correctness matters	DNS, streaming, VoIP, games — when speed/low latency matters more than occasional loss

End of solutions (Questions 1–3 as shown on the June 2025 paper).